Fixed-Size Memory vs. the Growing Cache: Outer-Product Fast Weights as an Alternative to KV-Caching

The problem

A standard Transformer answers a query by attending over every previous token, which requires keeping an explicit key–value (KV) cache that grows linearly with sequence length — memory and compute both scale with n. An alternative family of architectures avoids this by folding each new token into a **fixed-size state** instead of appending to a list. The mechanism that makes this possible — variously called a "fast weight," a "linear-attention state," or, in Pathway's Dragon Hatchling (BDH) architecture, a "synapse" — is the same operation in every case: an **outer-product update**. Given a key vector k and a value vector v, the state S (a fixed $d \times d$ matrix) is updated as `S ← S + k·vᵀ`, and later queried by `v̂ = k_qᵀS`. No new storage is allocated per token.

What changes technically, and what it costs

Replacing an append-only cache with a fixed-size accumulator changes memory from $O(n)$ to $O(d^2)$ — a real, measurable win for long sequences. The cost is that multiple associations now share the same d-dimensional capacity. For random keys, the readout for a queried item is (exactly, by linearity) the true value plus a noise term contributed by every other stored association; for random unit vectors this noise has expected squared norm $\approx (n-1)/d$, giving retrieval cosine similarity $\approx \sqrt{d/(d+n-1)}$ — a first-order approximation we derived and cross-checked against simulation (see the accompanying artifact's `verification/` folder; 30-seed average at d=16, n=64 was 0.418 vs. a predicted 0.450). A Transformer's cache never pays this cost — it degrades in memory, not accuracy. This is a real trade-off, not a free lunch on either side.

Where BDH and BDH-CQ fit

BDH (Pathway, arXiv:2509.26507) implements exactly this mechanism at neuron-synapse granularity: its working memory is a synaptic strength σ(i,j) updated during inference by σ(i,j) += Y(i)X(j) (§1.2) — an outer-product, Hebbian write the paper itself calls a "fast weight" and treats as equivalent to a linear-attention edge reweighting. The one structural difference from a generic fast-weight layer is that BDH's Y and X are sparse and non-negative (~5% of neurons active at a time, §6.4), which changes how quickly capacity fills but not the underlying mechanism. BDH-CQ (arXiv:2608.09888) reuses the identical accumulation principle one level up: its contextual-memory state accumulates additively as in-context demonstrations arrive, which the paper itself relates to fast-weight and linear-attention views of contextual association. **Important evidence caveat:** neither paper publishes a retrieval-accuracy-vs-capacity curve for its own synapse matrix — the degradation behavior described above is a property of the general outer-product mechanism class, demonstrated here with a simplified dense-vector toy model and cross-checked against classical linear-associative-memory theory, not a number BDH's authors measured.

Competitive context: three answers to "what happens at capacity"

Pure accumulation (BDH) is only one point in a small design space. **Titans** (arXiv:2501.00663, Google Research, Jan 2025) adds explicit forgetting via weight decay, trading long-term recall for reduced interference. **Gated DeltaNet-2** (arXiv:2605.22791, 2026) and **Variational Linear Attention** (arXiv:2605.11196, 2026) — both current research, not settled results — add error-corrective (delta-rule) updates specifically to reduce associative interference under multi-key recall, at the cost of extra per-step computation and a more complex update rule than pure Hebbian accumulation. None of these is strictly better: gating helps with recency-weighted recall but sacrifices BDH's simplicity and its order-independence (BDH's pure sum treats an old association and a new one identically; a gated model does not, by design). This is an active research trade-off, not a solved problem — as of writing, no architecture in this family has demonstrated that it dominates a full KV-cache on both memory *and* exactness simultaneously; the cache's advantage (no interference, ever) is structural, not a temporary engineering gap.

Evidence classification

- **Formally derived, independently verified here:** the interference/capacity trade-off itself (a mathematical consequence of finite-dimensional outer-product accumulation).
- **Reported by the developer, not independently reproduced here:** BDH's ~5% sparsity finding, BDH-CQ's 29.5%/$0.0007 ARC-AGI-1 result.
- **Not claimed:** that BDH or BDH-CQ have published the specific interference curve shown in the accompanying interactive artifact, or that any one of these four mechanisms is categorically superior to the others outside the specific axis (memory growth vs. interference) discussed here.

The open question

The most important unresolved piece is *consolidation*: how (or whether) a system built on fast, per-session accumulation converts durable, useful fast-state into permanent slow weights without simply retraining from scratch — an open problem BDH's own framing acknowledges rather than resolves.